Setting up a Python project

This is a tutorial showing you how to set up a Python project for data analysis and machine learning.

Step by step, we will enhance the capabilities of the project, starting from a minimal setup, ending with a fully functional project structure that you can use as a template for your own projects.

Features

We will increase the capabilities of the project step by step:

Major version 1 - basic project setup & version control with `git`

✅ Dedicated project folder with "everything" inside
✅ A `README.md` file as starting point for all project documentation
✅ Configuration of project and dependencies in `pyproject.toml`
✅ A Python virtual environment for dependency management (managed with `uv`)
✅ Script files for reusable code
✅ Jupyter notebooks for interactive data analysis
✅ Repository structure for data, notebooks, scripts (i.a.)
✅ Version control with `git`
✅ Synchronization with GitHub

Major version 2 - Introducing further best practices from Software Development

✅ Packaging the project as a Python package for easy reuse
✅ Logging with `logging` module (instead of print statements)
✅ Testing with `pytest`
✅ Linting of code with `ruff`

Major version 3 - Advanced project features

✅ Command-line interface with `Typer` to run analyses reproducibly
✅ Use `uv tool` for running the project's command-line interface
✅ Static type checking (e.g., with `mypy` or `Pyright`) for reusable modules
✅ Pre-commit hooks to run checks such as `ruff` before changes are committed
✅ Test coverage reporting with `pytest-cov` to identify untested code
✅ Automated dependency updates and vulnerability checks (e.g., Dependabot or Renovate)

Further enhancements can be added later, such as:

🔲 Data validation and schema checks (e.g., with `Pandera`) for incoming files.
🔲 Continuous integration with GitHub Actions to run tests, linting, formatting, and builds.
🔲 User and API documentation with `Sphinx`

Prerequisites

It is assumed that you have already installed the required software as described in the installation instructions in the readme.

Major Version 1 - basic project setup & version control with `git`

Version 1.0 - Initial project setup

Features

✅ Dedicated project folder with "everything" inside
✅ A `README.md` file as starting point for all project documentation
✅ Configuration of project and dependencies in `pyproject.toml`
✅ A Python virtual environment for dependency management (managed with `uv`)
✅ Script files for reusable code
🔲 Not yet: Version control with `git` - but we will already have a `.gitignore` template for typical files not to be synced with git.

1. Create a new folder for the project

Create a new folder for your project, e.g. `my-project` and open it in VS Code.

> **IMPORTANT Guidelines for choosing a project directory**: 1. you should store python projects on a local drive (**not on a network drive**) and avoid using special characters or spaces in the directory name.
>
> 1. Furthermore, things will run more smoothly, if you pick a folder that is **not synchronized with OneDrive** (e.g. usually `C:\Users\<you>\projects`. In contrast, `C:\Users\`

> `<you>\Documents\projects` might cause issues as the `Documents`, `Desktop` and similar folders are usually synchronized with OneDrive).

2. Initialize with `uv`

Open a terminal, then run:

```
uv init --python 3.14
```

This will create several files: - `.gitignore`: Specifies files and folders that should not be tracked by git, preconfigured for typical Python projects - `.python-version`: Specifies the Python version for the virtual environment - `main.py`: A simple Python script with a print statement - `README.md`: A markdown file for project documentation - `pyproject.toml`: Configuration file for the Python project and dependencies

Feel free to browse and inspect these files.

> **If `uv` is not available**: let me know; we can create the basic structure manually.

3. Edit the `pyproject.toml` file (e.g. project name, version)

4. Setup a minimal virtual environment

with uv:

Run the following command in the terminal to create a virtual environment with the Python version pinned in `.python-version` (Python 3.14; uv downloads it if needed):

```
uv venv
```

or, without uv:

```
python3 -m venv .venv
```

5. Activate the virtual environment

Activating the virtual environment (venv) differs between operating systems:

On Windows (PowerShell):

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.\.venv\Scripts\activate
```

On macOS or Linux:

```
source .venv/bin/activate
```

6. Run the main script

Run the main script with:

```
python main.py
```

7. *Optional*: Update the version in `pyproject.toml`

Edit the `pyproject.toml` file to set the version to `1.0`:

```
version = "1.0"
```

Version 1.1 - add jupyter notebooks

Features

In addition to the previous version, we add the following features:

✅ Jupyter notebooks for interactive data analysis

1. Add dependencies for running jupyter notebooks

Run the following command in the terminal:

```
uv add --dev ipykernel
```

or, without uv:

```
python -m pip install ipykernel
```

2. Create a folder for notebooks

Use the file explorer in VS Code to create a new folder `notebooks` or open a terminal in the project folder and run:

```
mkdir notebooks
```

3. Create a new notebook with a simple print statement

Use VS Code to create a new file `notebooks/analysis.ipynb` and add a code cell with the following content:

```
print("Hello, Jupyter!")
```

4. Run the notebook

Use the "Run" button in VS Code to run the code cell. You should see the output `Hello, Jupyter!` below the cell.

5. *Optional*: Update the version in `pyproject.toml`

Version 1.2 - add dependencies (i.e. pandas, openpyxl) and some data

---

Features

In addition to the previous version, we add the following features:

✅ Repository structure for data, notebooks, scripts (i.a.)

1. Add pandas as a dependency

Run the following command in the terminal:

```
uv add pandas openpyxl
```

or, without uv:

```
python -m pip install pandas openpyxl
```

This will add `pandas` as a dependency to the project and install it in the virtual environment.

2. Create a `data` folder with subfolders for different data processing stages

Use the file explorer in VS Code to create a new folder `data` with subfolders `raw`, `interim`, `prepared`, and `exports` or open a terminal in the project folder and run:

```
mkdir -p data/raw
mkdir -p data/interim
mkdir -p data/prepared
mkdir -p data/exports
```

3. Download some example data

Download the example data file from http://www.tobiaskeller.net/data/data_raw.zip and unzip it into the `data/raw` folder.

4. Load and inspect the data in a jupyter notebook

Create a new notebook `notebooks/data_inspection.ipynb` and add the following code to load and inspect the data:

```
import pandas as pd
df = pd.read_excel("../data/raw/wdi_reduced.xlsx", sheet_name="wdi")
df.head()
```

5. Go through the data-loading-checklist

Refer to the checklist for data loading issues if you encounter any issues loading the data.

6. *Optional*: Update the version in `pyproject.toml`

🧠 Live-Quiz on Kahoot 🏆

Go to https://www.kahoot.it and enter the game PIN to join the quiz.

Version 1.3 - Track your code with `git` and sync with GitHub

Features

In addition to the previous version, we add the following features:

✅ Version control with `git`
✅ Synchronization with GitHub

> 💡 **Tip (Reminder):** If you want your initial branch to be named `main` instead of the default (`master` in older Git versions), you can add:
>
> `bash`
> `git config --global init.defaultBranch main`
>
> This ensures consistency with platforms like GitHub, which default to `main`.

1. Initialize a git repository

Run the following command in the terminal:

```
git init
```

This will initialize a new git repository in the project folder.

2. Add the `data` folder to .gitignore

> 💡 **Note:** You should generally keep data files out of git!

In order to keep the data out of git, we add the `data` folder to the `.gitignore` file. This ensures that data files are not tracked by git. Add the following line to the file `.gitignore`:

```
# Ignore data files but keep folder structure
data/**
!data/**/.gitkeep
```

3. Add `.gitkeep` files to all data folders

In order to keep the folder structure in git, we add an empty file named `.gitkeep` to all data folders. This ensures that the folders are tracked by git, even if they are empty. Create an empty file named `.gitkeep` in each of the following folders.

You can use the terminal to create the files (commands differ between operating systems):

On Windows (PowerShell):

```
New-Item -Path data/raw/.gitkeep -ItemType File
New-Item -Path data/interim/.gitkeep -ItemType File
New-Item -Path data/prepared/.gitkeep -ItemType File
New-Item -Path data/exports/.gitkeep -ItemType File
```

On macOS or Linux:

```
touch data/raw/.gitkeep
touch data/interim/.gitkeep
touch data/prepared/.gitkeep
touch data/exports/.gitkeep
```

4. Add and commit all non-data files

> **Note:** Commit `pyproject.toml`, `uv.lock`, and `.python-version` for reproducibility.

Run the following commands in the terminal:

```
git add .
git add data/**/.gitkeep --force # on some systems, we need to enforce the
inclusion the first time
git commit -m "initial project setup"
git branch -M main # this is only necessary if you have not globally set the
default branch to main, as indicated above
```

5. Create a new repository on GitHub

Login to your GitHub account and create a new repository named `my-project` (or another name of your choice). Create a **bare** repository, i.e. **do not initialize** the repository with a README, .gitignore, or license.

6. Setup SSH keys for authentication (if not done yet)

If you have not set up SSH keys for authentication with GitHub yet, follow the instructions at https://docs.github.com/en/authentication/connecting-to-github-with-ssh to generate a new SSH key pair and add the public key to your GitHub account.

Alternatively, you can also use HTTPS for authentication, but you will have to enter your username and password (or personal access token) every time you push changes to GitHub.

7. *Optional*: Update the version in `pyproject.toml`

8. Add the remote repository and push the code

It is assumed you have created a new repository on GitHub named `my-project`. Replace `my-project` with the name of your repository and `XXXXXXXXX` with your GitHub username in the commands below.

Alternative 1: Using SSH (recommended)

Run the following commands in the terminal (replace `XXXXXXXXX` with your GitHub username):

```
git remote add origin git@github.com:XXXXXXXXX/my-project.git
git push -u origin main
```

Alternative 2: Using HTTPS

Run the following commands in the terminal (replace `XXXXXXXXX` with your GitHub username):

```
git remote add origin https://github.com/XXXXXXXXX/my-project.git
git push -u origin main
```

9. Demo: track changes with git using the VS Code source control pane

We will learn how to: * stage changes * commit changes * push changes to GitHub * make changes directly in GitHub and pull them to the local repository * view the history of changes using the `Git Graph` extension * revert to an earlier version of our code

Exercise 1: Setup your own project from scratch

1. Create a new, empty project folder
2. Set up the project as a Python package (hint: use `uv init` if available, edit `pyproject.toml` where necessary)
3. Set up a virtual environment with Python 3.14 and install `pandas` and `ipykernel` to add minimal dependencies for this project
4. Create a repository structure with folders for `data`, and `notebooks`.
5. Download the file "bank+marketing.zip". It contains (among others) the file `bank-additional-full.csv` with data from a marketing campaign of a Portuguese bank. Place that file in the `data/raw` folder.
6. Create a notebook that loads the data with pandas and inspects it (`pd.read_csv()`, note the delimiter is a ";", i.e. `pd.read_csv("PATHTOFILE", sep=";")`).
7. Initialize a git repository and sync it with a new GitHub repository in your account.
8. Commit and push your code to GitHub.

Major Version 2 - Introducing further best practices from Software Development

Version 2.0 - Packaging reusable code in modules

Features

In addition to the previous version, we add the following features:

✅ Packaging the project as a Python package for easy reuse

1. Create a `src` folder for the Python package

We are going to create a package named `my_project` (you can choose another name if you like). This enables you to reuse code in scripts and notebooks easily.

Use the file explorer in VS Code to create a new folder `src/my_project` or open a terminal in the project folder and run:

```
mkdir -p src/my_project
```

2. Create an empty `__init__.py` file in the package folder

Create an empty file `src/my_project/__init__.py`. This file indicates that the folder is a Python package.

3. Create a module `io.py` for data input/output functions

Create a new file `src/my_project/io.py` and add the following code:

```
"""Module for reusable input/output functions."""

import pandas as pd

def load_wdi_excel(path: str) -> pd.DataFrame:
    """Load the WDI Excel file into a DataFrame and rename some columns."""
    df = pd.read_excel(path, sheet_name="wdi")
    df = df.rename(
        columns={
            "NY_GDP_MKTP_CD": "gdp",
            "NY_GDP_MKTP_KD_ZG": "gdp_growth",
            "SP_POP_TOTL": "population"
        }
    )
    return df
```

4. Edit `pyproject.toml` to include the package

Edit the `pyproject.toml` file to include the package. Add the following lines at the end of the file:

```
[tool.uv]
package = true

[build-system]
requires = ["hatchling>=1.25"]
build-backend = "hatchling.build"
```

```
[tool.hatch.build.targets.wheel]
packages = ["src/my_project"]
```

or, if `uv` is not available:

```
[build-system]
requires = ["setuptools>=61.0", "wheel"]
build-backend = "setuptools.build_meta"
```

5. Install the package

> **Note:** Make sure to activate the virtual environment first!

Run the following command in the terminal:

```
uv sync
```

or, without uv:

```
python -m pip install -e .
```

6. Use the module in a jupyter notebook

Create a new notebook `notebooks/use_module.ipynb` and add the following code to use the `load_wdi_excel` function from the `my_project.io` module:

```
from my_project.io import load_wdi_excel

df = load_wdi_excel("../data/raw/wdi_reduced.xlsx")
df.head()
```

6. *Optional*: Update the version in `pyproject.toml`

7. *Optional*: build the package for distribution

In a previous step, we configured the project as a Python package. You can build the package for distribution with the following command:

```
uv build
```

This will create a `dist` folder with the package files that can be uploaded to PyPI or shared with others. When others download the `wheel` file, they can install it with `pip install <file>` or `uv add <file>`.

or, without uv:

```
python -m pip install build
python -m build
```

Version 2.1 - Add proper logging and a helper function for logging setup

> **Background:** It is considered best practice to use the built-in `logging` module instead of print statements for logging messages in Python applications. This allows for better control over the logging output and levels (e.g. debug, info, warning, error). Furthermore, in production code, print may leak sensitive information to the console. With `logging`, clients can configure logging handlers to redirect logs to files or other destinations, and set appropriate logging levels (see https://docs.astral.sh/ruff/rules/print/ for more details).

Features

In addition to the previous version, we add the following features:

✅ Logging with `logging` module (instead of print statements)

1. Try out a simple logger in a script

Create a new file `scripts/logging_example.py` and add the following code:

```
import logging

def main():
    logging.basicConfig(level=logging.INFO)
    logger = logging.getLogger(__name__)
    logger.info("This is an info message")
    logger.debug("This is a debug message")
    logger.warning("This is a warning message")
    logger.error("This is an error message")
```

```
    logger.critical("This is a critical message")

if __name__ == "__main__":
    main()
```

Run the script with `uv run`; this ensures that the virtual environment is activated automatically:

```
uv run python scripts/logging_example.py
```

Alternatively, activate the virtual environment yourself and run:

```
python scripts/logging_example.py
```

Note a few things: 1. We configure the logging level to `INFO`, so that debug messages are not shown. 2. We create a logger with `__name__`, so that the logger name corresponds to the module name. 3. There is no timestamp in the log message (which would be useful)

2. Improve the logging setup and change the log level to `DEBUG`

Edit the `scripts/logging_example.py` file to improve the logging setup:

```
import logging

def setup_logger(name: str, level: int | str = logging.DEBUG) -> logging.Logger:
    """Setup a logger with the given name and level."""
    logger = logging.getLogger(name)
    logger.setLevel(level)
    # Avoid duplicate handlers on re-import.
    if not any(
        isinstance(handler, logging.StreamHandler)
        and not isinstance(handler, logging.FileHandler)
        for handler in logger.handlers
    ):
        handler = logging.StreamHandler()
        handler.setFormatter(
            logging.Formatter("%(asctime)s - %(name)s - %(levelname)s - %
(message)s")
        )
        logger.addHandler(handler)
    logger.propagate = False
    return logger

def main():
```

```
    logger = setup_logger(__name__, level=logging.DEBUG)
    logger.info("This is an info message")
    logger.debug("This is a debug message")
    logger.warning("This is a warning message")
    logger.error("This is an error message")
    logger.critical("This is a critical message")

if __name__ == "__main__":
    main()
```

3. Move the `setup_logger` function to a helper module in our package

Create a new file `src/my_project/logging_helper.py` and move the `setup_logger` function there.

4. Use the `setup_logger` function in our script

Edit the `scripts/logging_example.py` file to import and use the `setup_logger` function from the `my_project.logging_helper` module:

```
from my_project.logging_helper import setup_logger

def main():
    logger = setup_logger(__name__, level="DEBUG")
    logger.info("This is an info message")
    logger.debug("This is a debug message")
    logger.warning("This is a warning message")
    logger.error("This is an error message")
    logger.critical("This is a critical message")

if __name__ == "__main__":
    main()
```

Run the script again with:

```
uv run python scripts/logging_example.py
```

or:

```
python scripts/logging_example.py
```

5. *Optional*: Update the version in `pyproject.toml`

Version 2.2 - Add testing with `pytest`

> **Background:** Testing is an important aspect of software development. It helps to ensure that the code works as expected and to catch bugs early. `pytest` is a popular testing framework for Python that makes it easy to write and run tests.

Features

In addition to the previous version, we add the following features:

✅ Testing with `pytest`

1. Add dependencies for testing

Run the following command in the terminal:

```
uv add --dev pytest
```

or

```
python -m pip install pytest
```

The `--dev` option adds `pytest` to the project's development dependency group, keeping it out of runtime dependencies. When using pip directly, also record test tools in `[dependency-groups].dev` in `pyproject.toml`.

2. Create a `tests` folder for test files

Use the file explorer in VS Code to create a new folder `tests` or open a terminal in the project folder and run:

```
mkdir tests
```

3. Create a test file for the `logging_helper` module

Create a new file `tests/test_logging_helper.py` and add the following code:

```
import logging
import pytest
from my_project.logging_helper import setup_logger

@pytest.mark.parametrize("level", [logging.INFO, "INFO"])
def test_setup_logger_emits(level, caplog):
    logger = setup_logger("test_logger_unique", level=level)
    assert logger.name == "test_logger_unique"
    assert isinstance(logger, logging.Logger)
    caplog.set_level(logging.INFO, logger=logger.name)
    logger.addHandler(caplog.handler)
    try:
        logger.info("hello")
    finally:
        logger.removeHandler(caplog.handler)
    assert "hello" in caplog.text
```

The helper sets `logger.propagate = False`, so pytest's root capture handler cannot see the record unless the test temporarily attaches `caplog.handler` directly to this logger.

4. Run the tests with `pytest`

Run the following command in the terminal:

```
uv run pytest
```

You should see output indicating that the test passed.

Again, you can also activate the virtual environment yourself and run:

```
pytest
```

5. Setup the test pane in VS Code

In VS Code, search for the "Testing" icon on the left sidebar (looks like a beaker) and click on it. Then click on "Configure Python Tests" and select `pytest`. This will create a `.vscode/settings.json` file with the following content:

```
{
    "python.testing.pytestEnabled": true,
    "python.testing.pytestArgs": [
```

```
        "tests"
    ],
    "python.testing.unittestEnabled": false
}
```

You can now run and debug tests directly from the test pane in VS Code.

6. *Optional*: Update the version in `pyproject.toml`

Version 2.3 - Add linting with `ruff`

> **Background:** Linting is the process of checking code for potential errors and enforcing coding standards. `ruff` is a fast and popular linter for Python that can help you to improve the quality of your code.

Features

In addition to the previous version, we add the following features:

✅ Linting of code with `ruff`

1. Add dependencies for linting

Run the following command in the terminal after activating the virtual environment:

```
uv add --dev ruff
```

or

```
python -m pip install ruff
```

2. Add a basic configuration for `ruff` in `pyproject.toml`

Edit the `pyproject.toml` file to add a basic configuration for `ruff`. Add the following lines at the end of the file:

```
[tool.ruff]
line-length = 100
```

3. Configure your VS code settings to indicate the line length limit

Edit the `.vscode/settings.json` file to add the following setting:

```
{
    "editor.rulers": [100],
    "editor.formatOnSave": true,
    "[python]": {
        "editor.defaultFormatter": "charliermarsh.ruff"
    },
    "editor.codeActionsOnSave": {
        "source.fixAll.ruff": "explicit"
    }
}
```

4. Run `ruff` to lint the code

Run the following command in the terminal:

```
uv run ruff check .
```

or, after activating the virtual environment:

```
ruff check .
```

You will notice many issues in the test files.

5. Add exceptions for the test files and for print statements in notebooks

Edit the `pyproject.toml` file to add exceptions for the test files and for print statements in notebooks. Add the following lines at the end of the file:

```
[tool.ruff.lint.per-file-ignores]
"notebooks/*" = [
    "T201", # print ok in notebooks
    ]
"tests/*" = [
    "S101",    # use of assert detected
    "PLR2004", # magic values OK in tests
    "SLF001",  # access to private members OK in tests
    "INP001",  # no __init__.py for tests as they are automatically identified
```

```
    "ANN201",  # tests don't return by design
    ]
```

6. Install the `Ruff` extension in VS Code and inspect linting issues in the code

Install the "Ruff" extension in VS Code (search for "Ruff" in the extensions pane and install it).

7. *Optional*: Setup auto-formatting on save in VS Code

With the "Ruff" extension installed, setup auto-formatting on save in VS Code by adding the following lines to the `.vscode/settings.json` file:

```
{
    ...

    "editor.codeActionsOnSave": {
        "source.fixAll.ruff": true
    },

    ...
}
```

8. *Optional*: Auto-Fix linting issues in the code

```
uv run ruff check . --fix
```

or, after activating the virtual environment:

```
ruff check . --fix
```

9. *Optional*: Manually fix remaining linting issues in the code

Manually fix remaining linting issues in the code (e.g. in the test files).

10. *Optional*: Update the version in `pyproject.toml`

Exercise 2: Improve your project from Exercise 1

Complete this exercise after working through Major Version 2 and before starting Major Version 3.

1. Create a module in `src/my_project/logging.py` that reuses the `setup_logger` function from above. Complete `pyproject.toml` and rerun `uv sync` if needed to make the package available.
2. Use the logger in a script to log a few messages.
3. Set up `ruff` for linting and fix any issues in your code.
4. Add tests for your logging module and run them with `pytest`.

Version 3.0 - Add advanced project features

> **Background:** The project already packages reusable code, tests it, and checks its style. In this version, we add a command-line interface, static type checking, coverage reports, automatic checks before commits, and automated dependency updates.

Features

In addition to the previous versions, we add the following features:

✅ Command-line interface with `Typer`
✅ Use `uv tool` to install and run the project's command-line interface
✅ Static type checking with `mypy`
✅ Pre-commit hooks for linting, formatting, and type checking
✅ Test coverage reporting with `pytest-cov`
✅ Automated dependency updates with Dependabot and GitHub security alerts

1. Add the dependencies

Run these commands from the project directory. The version pins make the example reproducible; `uv` also records the resolved dependency graph in `uv.lock`.

```
uv add "typer==0.27.2"
uv add --dev "mypy==2.3.1" "pandas-stubs==3.0.5.260914" "pre-commit==4.6.2"
"pytest-cov==7.1.0"
```

2. Add a command-line interface

Create `src/my_project/cli.py`. This command reuses the `load_wdi_excel` function from the earlier version and writes the prepared data to a CSV file:

```
from pathlib import Path
from typing import Annotated

import typer

from my_project.io import load_wdi_excel

app = typer.Typer(no_args_is_help=True)


@app.callback()
def main() -> None:
        """Prepare data files for analysis."""


@app.command()
def prepare_wdi(
        input_file: Annotated[
                Path,
                typer.Argument(exists=True, file_okay=True, dir_okay=False,
readable=True),
        ],
        output_file: Path,
) -> None:
        """Convert a WDI Excel workbook into a prepared CSV file."""
        data = load_wdi_excel(str(input_file))
        output_file.parent.mkdir(parents=True, exist_ok=True)
        data.to_csv(output_file, index=False)
        typer.echo(f"Wrote {len(data)} rows to {output_file}")
```

Register the command in `pyproject.toml` so it is available when the project is installed:

```
[project.scripts]
my-project = "my_project.cli:app"
```

Sync the project, then try the command's help. Place the example workbook in `data/raw/` first to run the conversion:

```
uv sync
uv run my-project --help
uv run my-project prepare-wdi data/raw/wdi_reduced.xlsx data/prepared/wdi.csv
```

3. Install the command with `uv tool`

To run the command once without installing it as a persistent tool, use `uv tool run`. The `--from .` option tells `uv` to use the package in the current project directory:

```
uv tool run --from . my-project --help
```

`uvx` is a shorthand alias for `uv tool run`, so this does the same thing:

```
uvx --from . my-project --help
```

To make the command available for repeated use from your shell, install it persistently in an isolated tool environment:

```
uv tool install .
my-project --help
my-project prepare-wdi data/raw/wdi_reduced.xlsx data/prepared/wdi.csv
```

> **How the four ways of running the CLI differ**
>
> - `uv tool install .` installs `my-project` and its runtime dependencies in a separate, uv-managed environment. The command is then available from your shell without entering this project's virtual environment. Because this is a separate installation, reinstall or upgrade it after changing the project code.
> - `uv tool run --from . my-project ...` runs the project command from an isolated tool environment without registering a persistent installation. `uvx --from . my-project ...` is the equivalent shorthand.
> - `uv run my-project ...` runs the command from this project's environment. `uv` prepares that environment from the project configuration and lockfile, so you do not need to activate it first. This is convenient during development because it uses the current project code.
> - Activating `.venv` and then running `my-project ...` also uses this project's environment. `uv sync` must have installed the project and its command entry point there first.

> Activation selects the environment for your shell and adds its scripts to `PATH`; it does not install the CLI by itself.

The tool installation is separate from the project's `.venv`. After changing the project code, refresh the installed tool with:

```
uv tool upgrade --reinstall example-project
```

4. Configure static type checking

Add the following configuration to `pyproject.toml`. It checks the reusable package in `src/` in strict mode, while leaving notebooks and test helpers out of the initial check. The project uses Python 3.14, so set both the package's minimum version and mypy's target to 3.14:

```
[project]
requires-python = ">=3.14"
```

Add the mypy configuration:

```
[tool.mypy]
python_version = "3.14"
strict = true
files = ["src/my_project"]
```

Run the type checker with:

```
uv run mypy src/my_project
```

`pandas` does not include the public stubs needed by mypy, so the development dependencies include `pandas-stubs`. Add parameter and return annotations to reusable functions, and resolve any reported issues before continuing.

5. Report test coverage

Add coverage reporting to the existing `[tool.pytest.ini_options]` table in `pyproject.toml`:

```
[tool.pytest.ini_options]
testpaths = ["tests"]
addopts = "--cov=my_project --cov-report=term-missing"
```

The report identifies lines not exercised by tests. For the complete test suite, enforce an 80% minimum with:

```
uv run pytest --cov-fail-under=80
```

Create tests for the CLI using Typer's `CliRunner` and a temporary workbook. This keeps the test independent of the real data files and checks both the generated CSV and invalid input paths.

6. Add pre-commit hooks

Create `.pre-commit-config.yaml` in the project root. These local hooks use the project's pinned tools through `uv`:

```
repos:
    - repo: local
        hooks:
            - id: ruff-check
                name: Ruff lint
                entry: uv run --locked ruff check --fix
                language: system
                types: [python]
            - id: ruff-format
                name: Ruff format
                entry: uv run --locked ruff format
                language: system
                types: [python]
            - id: mypy
                name: mypy
                entry: uv run --locked mypy
                args: [src/my_project]
                language: system
                pass_filenames: false
```

Install the hooks in this clone and run them against all files once:

```
uv run pre-commit install
uv run pre-commit run --all-files
```

After installation, the configured hooks run automatically when you make a Git commit. They do not create commits for you.

7. Enable automated dependency updates

Create `.github/dependabot.yml`:

```
version: 2
updates:
    - package-ecosystem: "uv"
        directory: "/"
        schedule:
            interval: "weekly"
```

Dependabot uses this file to propose version updates to `pyproject.toml` and `uv.lock`. In the GitHub repository settings, also enable Dependabot alerts and Dependabot security updates to receive vulnerability alerts and security-fix pull requests. The YAML file alone does not enable those repository-level security features.

8. Lock, sync, and run all checks

Update and verify the lockfile, then run the project's tests and checks:

```
uv lock
uv sync --locked
uv run pytest --cov-fail-under=80
uv run mypy src/my_project
uv run ruff check .
uv run pre-commit run --all-files
```

Inspect the Git changes, then use your usual Git workflow to stage and commit the completed version when it is ready.

🧠 Live-Quiz on Kahoot 🏆

Go to https://www.kahoot.it and enter the game PIN to join the quiz.

Exercise 3: Build a tested command-line analysis

Extend the project from Exercise 1 (after completing Exercise 2 and Major Version 3) with a reproducible analysis of the bank-marketing data. The CLI should create a summary of the subscription outcome in the `y` column of `bank-additional-full.csv`.

1. Add a reusable, fully typed function in `src/my_project/analysis.py`. It should read the semicolon-delimited input file and produce a summary with the number and percentage of subscribed (`yes`) and not-subscribed (`no`) customers.
2. Add a Typer command named `summarize-campaigns` that accepts input and output file paths and writes the summary to the requested output file. Register the command in `[project.scripts]`.
3. Test the analysis function and CLI using a small temporary CSV fixture. Check the summary values, output file, and behavior for a missing input file; tests should not depend on downloading the full dataset.
4. Run the CLI on the Exercise 1 dataset with `uv run my-project summarize-campaigns ...`. Also try `uv tool run --from . my-project summarize-campaigns ...` or its `uvx --from .` shorthand. Optionally install the command persistently with `uv tool install .` and run it from the shell.
5. Run strict type checking on `src/my_project`, and use `pytest-cov` to reach at least 80% test coverage for the package.
6. Configure pre-commit to run Ruff and mypy, install the Git hooks, and run them against all project files.
7. Configure Dependabot updates for the `uv` project and enable Dependabot alerts and security updates in the GitHub repository settings.
8. Run `uv lock`, `uv sync --locked`, the tests with the coverage threshold, mypy, Ruff, and pre-commit. When the checks pass, use your normal Git workflow to review and save the changes.